

1. Arquitectura general

El vertical de Data Science ha desarrollado un motor de recomendación basado en pesos que funciona de forma completamente independiente al resto de verticales. Es una API REST construida con FastAPI y SQLite que se comunica con Full Stack a través de tres endpoints bien definidos. No usa Machine Learning — es un sistema determinista que aprende del comportamiento real del usuario a través de sus reseñas.

Componente	Descripción
Lenguaje	Python 3.11
Framework API	FastAPI + Uvicorn
Base de datos	SQLite — gestionada por Data Science, independiente de Full Stack
Despliegue	Docker — imagen portable, se levanta con un solo comando
Endpoints	3 endpoints — GET / · POST /recomendaciones · POST /actualizar-pesos

2. Cómo funciona el motor

Sistema de pesos por tag

Cada usuario tiene asociados los 30 tags del catálogo, cada uno con un peso entre 0.0 y 1.0. Al registrarse, todos los pesos arrancan en 0.0. Solo evolucionan cuando el usuario deja una reseña. Tags bien valorados suben. Tags mal valorados bajan. Con el tiempo el perfil se personaliza automáticamente.

Fórmula de aprendizaje

```
# Cuando el usuario deja una reseña:

val_media = (score_servicio + score_ambiente + score_calidad_precio) / 3

val_norm = (val_media - 1) / 4 # escala 1-5 → 0.0-1.0

peso_nuevo = peso_actual × 0.9 + val_norm × 0.1

# Ejemplo – usuario valora un concierto con 5 estrellas:
# val_norm = (5-1)/4 = 1.0
# peso(conciertos): 0.5 → 0.5×0.9 + 1.0×0.1 = 0.55 ← sube

# Ejemplo – usuario valora una exposición con 2 estrellas:
# val_norm = (2-1)/4 = 0.25
# peso(arte): 0.5 → 0.5×0.9 + 0.25×0.1 = 0.475 ← baja
```

Cálculo del score de recomendación

Cuando el usuario pide recomendaciones, el motor recibe la lista de actividades de Full Stack, aplica dos filtros duros y calcula un score para cada candidata:

Paso	Descripción
Filtro 1 — time_slot	Descarta actividades cuya franja horaria no coincide con la solicitada (mañana / tarde / noche)
Filtro 2 — distancia	Descarta actividades a más de 40 km usando la fórmula de Haversine con lat/lng
Score	Media de los pesos del usuario para los tags que coinciden con los tags de la actividad
Resultado	Devuelve las top_n actividades ordenadas por score de mayor a menor (default: 5)

```
# Ejemplo de scoring:

# Usuario: conciertos=0.8, euskaldun=0.6, nocturno=0.4
# Actividad A tags: [conciertos, euskaldun, nocturno]

score_A = (0.8 + 0.6 + 0.4) / 3 = 0.60

# Actividad B tags: [conciertos, nocturno]

score_B = (0.8 + 0.4) / 2 = 0.60

# Actividad C tags: [arte, cultura] → usuario no tiene esos pesos
score_C = (0.0 + 0.0) / 2 = 0.00 ← no aparece en recomendaciones
```

3. Base de datos SQLite

Data Science gestiona su propia base de datos SQLite, completamente independiente de la PostgreSQL de Full Stack. Contiene tres tablas: los pesos de los usuarios, las actividades de Kulturklik y los comercios de Open Data Euskadi.

Tabla user_weights — pesos de los usuarios

Una fila por usuario con 30 columnas, una por tag. Los guiones se convierten a guiones bajos para ser válidos en SQL.

```
CREATE TABLE user_weights (
  user_id INTEGER PRIMARY KEY,
  after_party REAL DEFAULT 0.0,
  after_work REAL DEFAULT 0.0,
  conciertos REAL DEFAULT 0.0,
  euskaldun REAL DEFAULT 0.0,
  gastronomia REAL DEFAULT 0.0,
  nocturno REAL DEFAULT 0.0,
  ... (30 columnas en total)
);
```

Tabla activities — actividades Kulturklik

1.003 actividades reales cargadas desde la API de Kulturklik del Gobierno Vasco. Estructura alineada con la tabla activities de Full Stack, con campos adicionales imprescindibles para el motor.

```
CREATE TABLE activities (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  name TEXT, -- nombre del evento  
  municipio TEXT, -- para filtros de zona  
  lat REAL, -- imprescindible para Haversine  
  lng REAL, -- imprescindible para Haversine  
  tag_1 TEXT, -- imprescindible para el scoring  
  tag_2 TEXT,  
  tag_3 TEXT,  
  time_slot TEXT, -- mañana | tarde | noche  
  price REAL, -- precio fijo (alineado con Full Stack)  
  fecha_inicio TEXT,  
  fecha_fin TEXT,  
  is_indoor INTEGER, -- alineado con Full Stack  
  is_active INTEGER -- alineado con Full Stack  
);
```

Tabla businesses — comercios Open Data Euskadi

1.720 restaurantes y comercios reales cargados desde Open Data Euskadi con nombre, localidad, coordenadas, tipo y tres tags.

Tabla	Origen de datos	Filas
user_weights	Generados con script seed.py (25 usuarios sintéticos)	25
activities	API Kulturklik — Gobierno Vasco	1.003
businesses	Open Data Euskadi — restaurantes y comercios	1.720

```
# Cargar los datos reales en SQLite:  
  
python load_data.py  
  
# Pobl原因 usuarios sintéticos para pruebas:  
  
python seed.py
```

4. Endpoints de la API

GET /	
Health check. Verifica que el servicio está activo.	
Sin body	DEVUELVE { "status": "ok", "service": "Motor de Recomendación" }

POST /recomendaciones	
Recibe el perfil del usuario y la lista de actividades. Filtra por time_slot y distancia (40 km), calcula scores y devuelve top 5.	
RECIBE user_id, lat, lng, time_slot, top_n (opcional) activities: lista con id, name, tags, time_slot, lat, lng	DEVUELVE Lista de actividades con activity_id, name, score, distancia_km Ordenadas por score de mayor a menor

POST /actualizar-pesos	
Recibe las puntuaciones de una reseña. Calcula la media, actualiza los pesos del usuario en SQLite y los devuelve.	
RECIBE user_id, activity_tags[] score_servicio, score_ambiente, score_calidad_precio (1-5)	DEVUELVE ok: true weights: JSON con los 30 tags y sus pesos actualizados val_norm: valoración normalizada usada en el cálculo

5. Despliegue con Docker

El servicio se dockeriza para que Full Stack pueda levantarlo con un solo comando sin instalar dependencias ni configurar entornos. La BD SQLite se monta en un volumen para persistir los pesos entre reinicios del contenedor.

```
# Construir la imagen Docker
docker build -t motor-recomendador .

# Levantar el contenedor
# -v monta volumen para persistir weights.db entre reinicios
# -p expone el puerto 8000
docker run -v ./data:/data -p 8000:8000 motor-recomendador

# Documentación automática Swagger:
http://localhost:8000/docs
```

Archivo	Descripción
main.py	API FastAPI — motor completo con los 3 endpoints
Dockerfile	Imagen Docker basada en Python 3.11 slim

requirements.txt	fastapi, uvicorn, pydantic (SQLite viene en Python)
seed.py	Pobla la BD con 25 usuarios sintéticos para pruebas
load_data.py	Carga las actividades y comercios reales desde los CSVs
weights.db	BD SQLite — se genera automáticamente, no se sube al repo

6. Separación de responsabilidades entre verticales

Qué	Full Stack (PostgreSQL)	Data Science (SQLite)
Usuarios	Registro, login, autenticación	—
Preferencias	Tags seleccionados por el usuario	Pesos de los 30 tags (evolución)
Actividades	CRUD completo, formularios	Copia local para el motor de scoring
Comercios	CRUD completo, formularios	Copia local para contexto
Reseñas	Guarda en reviews y llama a /actualizar-pesos	Actualiza pesos en user_weights
Recomendaciones	Llama a /recomendaciones con las activities	Calcula scores y devuelve top 5

Nota clave: Data Science puede desarrollar y probar el motor de forma completamente autónoma con los datos sintéticos del seed y los datos reales de Kulturklik y Open Data Euskadi, sin depender de que Full Stack tenga sus endpoints listos. La integración final solo requiere conectar los tres endpoints.